NFTmarket

A Complete Guide for Watermarking Experts

How Invisible Watermarks Meet Blockchain Technology to Enable Copyright Trading

Watermark
+
Blockchain

NFTmarket Documentation

February 6, 2026

Contents

I Foundation Concepts **2**

1 The Problem We're Solving **2**
1.1 The Four Problems NFTmarket Solves . . . 2
1.2 The Solution . . . 2

2 Blockchain Fundamentals **3**
2.1 What Is a Blockchain? . . . 3
2.2 Key Properties . . . 3
2.3 Why This Matters for Art . . . 3

3 What Is an NFT? **4**
3.1 What "Non-Fungible" Means . . . 4
3.2 What the NFT Points To . . . 4
3.3 The Standard Interface (ERC-721) . . . 4

4 Smart Contracts: The Database on the Blockchain **5**
4.1 What NFTmarket's Contract Stores . . . 5
4.2 Key Functions . . . 5
4.3 Why Contracts Are Powerful . . . 5

5 Wallets: Your Digital Identity **6**
5.1 Public Key vs Private Key . . . 6
5.2 How Transactions Work . . . 6
5.3 Sign-In with Ethereum (SIWE) . . . 6

II The NFTmarket System **7**

6 The NFTmarket Architecture **7**
6.1 The Services . . . 7
6.2 Data Flow . . . 7
6.3 Why a Separate Wallet Service? . . . 8

7 The License System: Selling Copyright **9**
7.1 The Three License Types . . . 9
7.2 License Hierarchy . . . 9
7.3 What Each License Allows . . . 9
7.4 Multi-License Marketplace . . . 10
7.5 Bilateral Signatures . . . 10

8 The Minting Flow: From Image to NFT **11**
8.1 The Token ID Problem . . . 11
8.2 The Solution: Predict (or Reserve) the Token ID . . . 11
8.3 Complete Minting Flow . . . 11
8.4 What Gets Embedded in the Watermark . . . 12
8.5 What Gets Stored Where . . . 12
8.6 Both Copies Are Watermarked . . . 12

9 Buying and Selling Tokens **13**

9.1 Listing Flow . . . 13
9.2 Buying Flow . . . 13
9.3 License Rights Transfer . . . 13
9.4 Payment Distribution . . . 14

III The Watermark-Blockchain Integration 15

10 How Watermarks Enable Copyright Trading 15
10.1 The Problem with Regular NFTs . . . 15
10.2 The Solution: Embedded Identity . . . 15
10.3 Why Both Wallet AND Token ID? . . . 15
10.4 The Verification Chain . . . 15

11 The Dual-Proof System 17
11.1 Layer 1: The Watermark (Embedded in Pixels) . . . 17
11.2 Layer 2: The Blockchain (Immutable Record) . . . 17
11.3 Why Both Layers Matter . . . 17

12 Protecting Original Images 18
12.1 Before Minting: Artist Responsibility . . . 18
12.2 Best Practices for Artists . . . 18
12.3 The Blockchain Timestamp Is Your Proof . . . 18

13 Fraud Detection and Prevention 19
13.1 What the Watermark Protects Against . . . 19
13.2 What the Watermark Does NOT Protect Against . . . 19
13.3 If You Buy a Fraudulent Token . . . 19
13.4 Best Protection for Artists . . . 19

IV Technical Deep Dive 20

14 Storage Architecture 20
14.1 Where Everything Lives . . . 20
14.2 Watermark-Relevant Storage . . . 20
14.3 Why IPFS for Preview, Not for Encrypted Original? . . . 20

15 The Central Point of Failure 21
15.1 If NFTmarket Disappears . . . 21
15.2 Why There's No "Publish the Spec" Escape Hatch . . . 21
15.3 What Still Works Without the Authority . . . 21
15.4 What Dies Without the Authority . . . 21
15.5 IPFS Failure vs NFTmarket Failure . . . 22

16 Copyright and Legal Reality 23
16.1 AI Art Cannot Be Copyrighted . . . 23
16.2 What the Watermark Actually Provides . . . 23
16.3 The Opportunity for Physical Artists . . . 23

17 Watermark Technical Details 24
17.1 The Watermarking API . . . 24
17.2 Watermark Properties . . . 24

17.3 The Robustness Trade-off 24

18 Future Directions: Zero-Knowledge Watermarks **25**
18.1 The Vision 25
18.2 How It Could Work 25
18.3 The Challenges 25
18.4 Current Status 25

Summary: Key Takeaways **26**

Part I

Foundation Concepts

1 The Problem We're Solving

Traditional NFTs have a fundamental problem: **buying an NFT doesn't transfer any actual rights**.

When someone buys a typical NFT, they receive:

- A database entry saying they "own" a token
- That's it

They do NOT receive:

- Copyright
- The right to reproduce the image
- The right to make merchandise
- Any legal ownership of the artwork itself

Anyone can right-click, save the image, and use it however they want. The NFT buyer has nothing but a receipt.

1.1 The Four Problems NFTmarket Solves

1. **NFTs don't convey ownership** – Buying an NFT doesn't transfer copyright or usage rights
2. **AI art has no provenance** – Anyone can generate similar images; no way to prove who created first
3. **Stolen art is rampant** – Images are re-minted without consequences
4. **Physical art lacks accessible authentication** – Expensive gallery systems, easily forged certificates

1.2 The Solution

NFTmarket combines **invisible watermarking** + **NFT minting** + **legal licensing** into a single transaction.

The watermark embeds the creator's identity directly into the pixels. The blockchain provides an immutable timestamp and ownership record. The license agreement defines exactly what rights are being sold.

This is where your watermarking expertise becomes essential. The watermark is the bridge between the digital image and the blockchain record. Without it, there's no way to connect a JPEG floating around the internet back to its token.

2 Blockchain Fundamentals

Before we can understand NFTs, we need to understand what they live on: a blockchain.

2.1 What Is a Blockchain?

A blockchain is a **distributed database** replicated across thousands of computers worldwide. Every participant has an identical copy of all the data.

Traditional Database	Blockchain
Runs on one company's servers	Runs on thousands of independent computers
Company controls it	No single party controls it
Company can modify data	Data is immutable once written
Trust the company	Trust the math

Table 1: Traditional Database vs Blockchain

2.2 Key Properties

Immutability: Once data is written, it cannot be changed. Every "block" of transactions is cryptographically linked to the previous block. Changing old data would require re-computing every subsequent block, which is computationally infeasible.

Transparency: Everyone can see every transaction. The entire history of the database is public.

Timestamps: Each block has a timestamp that cannot be forged. You can prove exactly when something was recorded.

No Central Authority: No company, government, or individual controls the blockchain. It's maintained by a decentralized network of participants.

2.3 Why This Matters for Art

For the first time, we have a public, permanent, tamper-proof record of:

- Who created something (and when)
- Who currently owns it
- Every transfer of ownership in between

This is the **provenance chain** – the unbroken history from creation to the current owner.

3 What Is an NFT?

An NFT (Non-Fungible Token) is surprisingly simple. It's just a **row in a database**.

```
Smart Contract Database
 Token #0 → owned by 0xAlice
 Token #1 → owned by 0xBob
 Token #2 → owned by 0xAlice
 Token #3 → owned by 0xCharlie
```

That's literally it. An NFT is an entry in a database that says "token X is owned by address Y."

3.1 What "Non-Fungible" Means

Non-fungible means non-interchangeable – each token has a unique identity.

- **Fungible:** One dollar bill equals any other dollar bill (interchangeable)
- **Non-Fungible:** Token #0 ≠ Token #1 (each is unique)

Two NFTs can have the same market price. "Non-fungible" just means they're tracked individually and can't be swapped blindly.

3.2 What the NFT Points To

The NFT itself doesn't contain the image. Storing images on the blockchain is prohibitively expensive.

Instead, the NFT contains a **pointer** (a URI) to where the image and metadata are stored:

```
Token #123
    → tokenURI: "ar://abc123..."  (Arweave permanent storage)
        → image: "ipfs://Qm..."   (IPFS decentralized storage)
        → name: "My Artwork"
        → description: "..."
```

The ownership lives on the blockchain. The data lives off-chain but is verifiable.

3.3 The Standard Interface (ERC-721)

All NFT contracts implement the same standard functions:

- `ownerOf(tokenId)` – Who owns this token?
- `transferFrom(from, to, id)` – Transfer ownership
- `balanceOf(address)` – How many tokens does this address own?

This standardization allows any marketplace or wallet to interact with any NFT.

4 Smart Contracts: The Database on the Blockchain

A smart contract is a program that runs on the blockchain. Think of it as:

Database + Code + Guaranteed Execution

Traditional App	Smart Contract
Database (like Postgres)	State variables (mappings, structs)
Backend code (like Node.js)	Solidity functions
Runs on your server	Runs on the blockchain
You control it	No one controls it

Table 2: Traditional App vs Smart Contract

4.1 What NFTmarket's Contract Stores

The NFTmarketNFT smart contract stores:

```
struct TokenData {
    address creator;           // Original artist (never changes)
    address copyrightOwner;    // Current copyright holder
    LicenseType ownerLicense;  // Display, Commercial, or Transfer
    bytes32 imageHash;         // SHA-256 of the watermarked image
    bytes32 licenseHash;       // Hash of signed license agreement
    uint256 mintedAt;          // Block timestamp (immutable)
    string encryptedBlobUri;   // Where encrypted original is stored
}
```

4.2 Key Functions

Function	What It Does
`mint()`	Create new NFT with license type
`list(tokenId, price, licenseType)`	List for sale
`buy(tokenId)`	Purchase (with payment)
`delist(tokenId)`	Remove listing
`ownerOf(tokenId)`	Query current owner
`getTokenData(tokenId)`	Query all metadata

Table 3: Smart Contract Key Functions

4.3 Why Contracts Are Powerful

Once deployed, the code is **permanent and trustless**:

- No one can change the rules after deployment
- The contract executes exactly as written
- Everyone can verify the code
- No company can shut it down or change behavior

5 Wallets: Your Digital Identity

A wallet is how users interact with the blockchain. Despite the name, it doesn't "store" money – it stores **cryptographic keys**.

5.1 Public Key vs Private Key

Key Type	Purpose	Visibility
Public Key (Address)	Your identity, like an email	Share freely: `0x742d35Cc...`
Private Key	Proves you own the address	**NEVER share**

Table 4: Public vs Private Keys

The address is derived mathematically from the private key. Anyone can send you things if they know your address. Only you can send things away (because only you have the private key).

5.2 How Transactions Work

Every blockchain transaction requires a **signature** from the sender's private key:

1. User creates transaction: "Send Token #42 to 0xBob"
2. User's wallet signs it with private key
3. Transaction broadcast to network
4. Network verifies signature matches address
5. Transaction executed and recorded forever

5.3 Sign-In with Ethereum (SIWE)

Instead of username/password, NFTmarket uses wallet signatures for authentication:

1. Server sends a challenge message
2. User signs it with their wallet
3. Server verifies the signature
4. User is authenticated

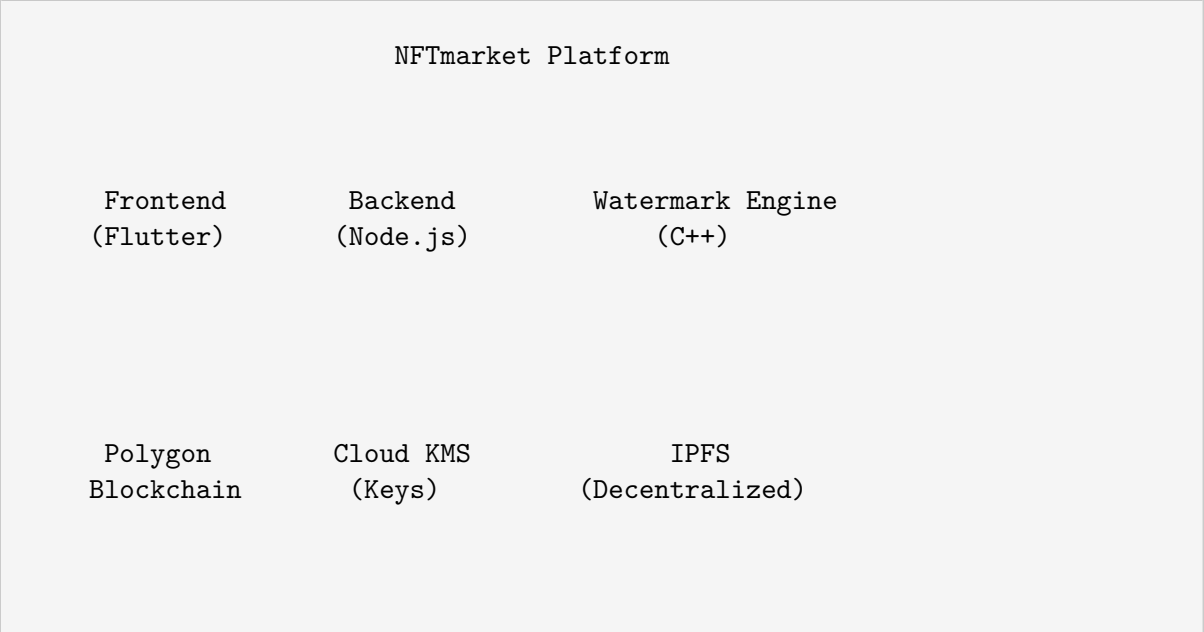
No passwords to steal. No accounts to hack. Your identity is your cryptographic key.

Part II

The NFTmarket System

6 The NFTmarket Architecture

NFTmarket consists of four independent components that communicate via API and blockchain:



6.1 The Services

Service	Port	Purpose
Flutter Web App	–	User interface (browser-based)
Backend API	3000	Main application server
Wallet Service	3001	Key management and transaction signing
Hardhat Node	8545	Local blockchain for development

Table 5: NFTmarket Services

6.2 Data Flow

```
User uploads image
       ↓
Backend: Apply watermark (wallet + tokenId)
       ↓
Backend: Encrypt original, upload preview to IPFS
       ↓
Backend: Upload metadata to Arweave (permanent)
       ↓
Backend: Store encrypted blob in Firebase Storage
       ↓
Wallet Service: Sign and submit mint transaction
       ↓
```

```
Blockchain: Record ownership, store hashes
       ↓
User: Owns NFT with embedded provenance
```

6.3 Why a Separate Wallet Service?

The Flutter web app **never touches private keys**. All signing happens in a separate microservice. This architecture:

- Prevents key leakage through the frontend
- Allows different key management strategies
- Enables server-side signing for automated operations
- Keeps sensitive operations isolated

7 The License System: Selling Copyright

This is where NFTmarket fundamentally differs from regular NFTs. Instead of just selling tokens, we sell **actual legal rights**.

7.1 The Three License Types

Type	What Buyer Gets	What Seller Keeps	Price
Display	Token + personal display rights	Copyright, commercial rights	$1
Commercial	Token + commercial usage rights	Copyright	$5
Transfer	Token + full copyright	Display rights only	$10

Table 6: License Types and Rights

7.2 License Hierarchy

Rights form a strict hierarchy. **You cannot sell more rights than you have.**

```
Creator mints with Transfer → can sell Display, Commercial, or Transfer
Buyer gets Commercial       → can only resell Display or Commercial
Buyer gets Display          → can only resell Display
```

7.3 What Each License Allows

Display License:

- Personal use only
- Can display in your home
- Can use as profile picture
- Cannot monetize
- Cannot make merchandise
- Cannot create derivatives

Commercial License:

- Everything in Display, plus:
- Can monetize (sell prints, merchandise)
- Can use in commercial projects
- Can create derivatives
- Cannot transfer copyright to someone else

Transfer License:

- Full copyright transfer
- Can do anything the original creator could do
- Can grant new licenses

- Previous owner retains only personal display rights

7.4 Multi-License Marketplace

A single token can have **up to three simultaneous listings** – one for each license type:

```
Token #42 Listings:
 Display:    0.5 ETH (personal use only)
 Commercial: 2.0 ETH (can make merch)
 Transfer:   10.0 ETH (full copyright)
```

Buyers choose their license level. The creator can earn from the same artwork multiple times at different price points.

7.5 Bilateral Signatures

Both buyer and seller sign the **exact same license agreement**:

```
   SELLER                  BUYER

 Signs:                 Signs:
 - tokenId              - tokenId
 - price                - price
 - license              - license
 - hash                 - hash




            BilateralAgreement
            Event (on-chain)

           Both signatures
           recorded forever
```

The license hash ensures both parties agreed to **exactly the same terms**. This creates a legally binding record on the blockchain.

8 The Minting Flow: From Image to NFT

This is where watermarking integrates with the blockchain. Understanding this flow is essential.

8.1 The Token ID Problem

Here's a chicken-and-egg problem:

- The watermark needs the token ID
- But the token ID is assigned during minting
- And minting requires the watermarked image first

```
Watermark needs tokenId
    ↓
Preview needs watermark
    ↓
Metadata needs preview URI
    ↓
Mint needs metadata URI
    ↓
Mint returns tokenId  ← too late!
```

8.2 The Solution: Predict (or Reserve) the Token ID

Token IDs are assigned sequentially. Before minting, we query `totalSupply()` and predict the next ID will be `currentSupply + 1`.

A more robust solution uses **reservation**:

```
Old flow (race condition risk):
  totalSupply() → predict ID → watermark → mint → hope IDs match

New flow (guaranteed):
  reserveTokenId() → get ID → watermark → mint(reservedId) → always match
```

8.3 Complete Minting Flow

1. User uploads image
2. Reserve tokenId on-chain (or predict it)
3. **Apply watermark: embed (wallet + tokenId)**
4. Calculate imageHash (SHA-256 of watermarked image)
5. Generate preview (800×800, FROM watermarked image)
6. Encrypt watermarked original (AES-256)
7. Upload preview to IPFS (public)
8. Upload encrypted blob to Firebase Storage
9. Create metadata JSON
10. Upload metadata to Arweave (permanent)

11. Mint NFT on Polygon (verify token ID matches)
12. Store mapping in Firestore

8.4 What Gets Embedded in the Watermark

```
Format: "0x742d35Cc6634C0532925a3b844Bc9e7595f8fE0E:123"
         wallet address   tokenId
```

Both pieces are needed:

- **Wallet address**: Proves who created it
- **Token ID**: Identifies exactly which NFT it is

8.5 What Gets Stored Where

Data	Location	Purpose
Token ownership	Blockchain	Source of truth
Mint timestamp	Blockchain	Immutable proof of priority
creator, copyrightOwner	Blockchain	Rights tracking
Preview (watermarked)	IPFS	Public display, traceable
Metadata JSON	Arweave	Permanent record
License documents	Arweave	Legal agreements
Encrypted original	Firebase Storage	Watermark detection
Encryption key	Cloud KMS	Controlled access
Watermark ID	Firestore + Metadata	Links image to token

Table 7: Data Storage Locations

8.6 Both Copies Are Watermarked

Image	Watermarked?	Where
Preview/thumbnail (800×800)	**Yes**	IPFS (public)
Full-resolution original	**Yes**	Firebase Storage (encrypted)

Table 8: Watermarked Image Copies

The preview is generated FROM the watermarked image. Screenshots of the preview carry the watermark and can be traced back to the token.

9 Buying and Selling Tokens

9.1 Listing Flow

1. **Seller:** Choose token, price, license type
2. **Seller:** Read license agreement text
3. **Seller:** Sign license agreement (EIP-712 typed data)
4. **Seller:** Call `contract.list()` with signature
5. **Contract:** Verify seller owns token
6. **Contract:** Verify seller has rights to sell this license type
7. **Contract:** Verify seller's signature
8. **Contract:** Store listing (price, license, signature, expiry)
9. **Contract:** Emit Listed event

9.2 Buying Flow

1. **Buyer:** Browse listings, find token
2. **Buyer:** Read license agreement (same one seller signed)
3. **Buyer:** Sign purchase acceptance (EIP-712)
4. **Buyer:** Call `contract.buy()` with signature + payment
5. **Contract:** Verify listing is active and not expired
6. **Contract:** Verify payment is sufficient
7. **Contract:** Verify buyer's signature
8. **Contract:** Transfer license rights (based on license type)
9. **Contract:** Distribute payment (97.5% seller, 2.5% platform)
10. **Contract:** Transfer NFT ownership
11. **Contract:** Emit Sold + BilateralAgreement events

9.3 License Rights Transfer

Display Sale:

```
Before: Owner=Seller, Copyright=Seller, License=Display/Commercial/Transfer
After:  Owner=Buyer, Copyright=Seller, License=Display
```

Commercial Sale:

```
Before: Owner=Seller, Copyright=Seller, License=Commercial/Transfer
After:  Owner=Buyer, Copyright=Seller, License=Commercial
```

Transfer Sale (Copyright Transfer):

```
Before: Owner=Seller, Copyright=Seller, License=Transfer
After:  Owner=Buyer, Copyright=Buyer, License=Transfer
        Seller gets hasDisplayLicense[seller] = true (moral rights)
```

9.4 Payment Distribution

Recipient	Amount (1 ETH sale)
Platform (2.5%)	0.025 ETH
Seller (97.5%)	0.975 ETH

Table 9: Payment Distribution

Part III

The Watermark-Blockchain Integration

10 How Watermarks Enable Copyright Trading

This is the crucial section. Here's why your watermarking expertise is the foundation of the entire system.

10.1 The Problem with Regular NFTs

With a standard NFT:

1. Someone right-clicks and saves your image
2. They repost it, crop it, screenshot it, share it
3. There is **zero connection** between that JPEG and your NFT
4. You can't prove it's yours. They can't prove it's not.

10.2 The Solution: Embedded Identity

With NFTmarket:

1. The watermark embeds `wallet:tokenId` into the pixels
2. Someone right-clicks and saves your image
3. The watermark **travels with the copy**
4. Screenshots, downloads, reposts – all carry the watermark
5. Anyone can extract the watermark and query the blockchain
6. Immediate proof of which token, who created it, who owns rights

10.3 Why Both Wallet AND Token ID?

The watermark must contain **both pieces**:

Embedded Data	Why It's Needed
Wallet address	Proves who created it
Token ID	Identifies exactly which NFT

Table 10: Watermark Data Requirements

With just the wallet: You know who made something, but they might have minted 100 NFTs. Which one?

With just the token ID: Token #42 could exist on any of thousands of contracts. Which one?

Together: Unique identification across the entire ecosystem.

10.4 The Verification Chain

```
Suspicious Image (JPEG, screenshot, whatever)
       ↓
Extract watermark → get wallet + tokenId
       ↓
Query blockchain for that token
       ↓
Confirm creator matches wallet from watermark
       ↓
Check copyrightOwner → who has rights now?
       ↓
Check mint timestamp → who minted first?
```

The watermark is the sole link between an image and its token.

11 The Dual-Proof System

NFTmarket uses two independent systems that serve different purposes:

11.1 Layer 1: The Watermark (Embedded in Pixels)

The watermark **identifies the token**. It's baked into the file and travels with any copy.

Property	Description
Survives	Screenshots, downloads, JPEG compression, resizing
Can be extracted from	Any copy of the image
Contains	Creator wallet address + token ID

Table 11: Watermark Properties

11.2 Layer 2: The Blockchain (Immutable Record)

The blockchain **proves ownership**. It's an external, permanent record.

Data	Description
`creator`	Original artist (never changes)
`copyrightOwner`	Who holds current rights (changes on Transfer sales)
`mintedAt`	Block timestamp proving when created

Table 12: Blockchain Data

11.3 Why Both Layers Matter

Watermark	Blockchain
Identifies which token	Records who owns it
Survives image distortion	Immutable ownership history
Embedded in pixels	External record
The only link after transformation	Source of truth for rights

Table 13: Watermark vs Blockchain Roles

If someone reposts your art without permission:

1. Extract the watermark → get wallet + token ID
2. Query the blockchain for that token → confirm creator, check current owner
3. You have proof of which token it is and who owns the rights

12 Protecting Original Images

12.1 Before Minting: Artist Responsibility

The unwatermarked original is a secret. Whoever has it can mint it first.

There is **no automatic duplicate detection**. If someone gets your original before you mint:

1. They upload it
2. They mint it
3. Their token has the earlier timestamp
4. They "own" it according to the system

12.2 Best Practices for Artists

Do	Don't
Keep originals on encrypted storage	Post originals on social media
Mint first, share later	Send originals to "collaborators"
Share only watermarked versions	Store in cloud with weak passwords

Table 14: Artist Best Practices

12.3 The Blockchain Timestamp Is Your Proof

Every mint is recorded with an immutable timestamp:

```
Token #42: Block 18,234,567 (January 15, 2025, 14:32:01 UTC)
Token #99: Block 18,300,000 (January 20, 2025, 09:15:44 UTC)
```

The earlier timestamp wins. This cannot be forged.

If two people mint the same image, the one with the earlier blockchain timestamp has priority. The timestamp is proof that will hold up in any dispute.

13 Fraud Detection and Prevention

13.1 What the Watermark Protects Against

If someone takes an **already-watermarked image** (screenshot, download, etc.) and tries to mint it:

1. The original watermark is still embedded (original `wallet:tokenId`)
2. Detection via `/api/detect` reveals the original token
3. The fraudster's token has a later timestamp
4. Proof of fraud is clear

13.2 What the Watermark Does NOT Protect Against

If someone obtains the **original unwatermarked image** before minting:

Scenario	Outcome
Artist mints first, fraudster second	Artist has earlier timestamp = proof of fraud
Fraudster mints first with stolen original	Fraudster has earlier timestamp. Artist looks like fraud.

Table 15: Fraud Scenarios

When a fraudster mints an original image, the system watermarks it with the **fraudster's** `wallet:tokenId`. There's no competing watermark to detect. The only evidence is timestamp comparison.

13.3 If You Buy a Fraudulent Token

Bad luck.

The buyer has no automatic recourse. Options:

1. **Compare timestamps** – Find the earlier token with the same image
2. **Sue the seller** – The seller's wallet address is on-chain
3. **Social pressure** – Public exposure of the fraudster's wallet

The platform does not offer refunds or fraud protection. **Buyer beware.**

13.4 Best Protection for Artists

Mint before your original leaks. Once minted, the watermark and timestamp protect you. Before minting, there's no protection.

Part IV

Technical Deep Dive

14 Storage Architecture

14.1 Where Everything Lives

Storage	Type	What It Stores	Durability
Blockchain (Polygon)	Decentralized	Ownership, timestamps, hashes	Permanent
Arweave	Permanent	Metadata, licenses	200+ years
IPFS (Pinata)	Decentralized	Preview images	Requires pinning
Firebase Storage	Centralized	Encrypted originals	Depends on NFTmarket
Cloud KMS	Centralized	Encryption keys	Depends on NFTmarket
Firestore	Centralized	Token-to-key mappings	Depends on NFTmarket

Table 16: Storage Architecture Overview

14.2 Watermark-Relevant Storage

What	Where	Purpose
Watermarked preview	IPFS (public)	Display, contains watermark
Watermarked original	Firebase (encrypted)	Detection by authority
Encryption key	Cloud KMS	Controlled access
Watermark ID	Firestore	Links image to token
Mint timestamp	Blockchain	Immutable proof of priority

Table 17: Watermark-Related Storage

14.3 Why IPFS for Preview, Not for Encrypted Original?

The encrypted blob is useless without the key. Since key management is centralized anyway, putting the blob on "decentralized" IPFS would be theater – you still depend on NFTmarket to decrypt it.

Firebase Storage is simpler, cheaper, and honest about what's centralized.

15 The Central Point of Failure

This is the honest assessment. The watermarking authority is essential and irreplaceable.

15.1 If NFTmarket Disappears

Component	Status	Impact
Blockchain records	**Survives**	Ownership, timestamps intact
Arweave metadata	**Survives**	Name, description permanent
Arweave licenses	**Survives**	Legal agreements verifiable
IPFS previews	**At risk**	Need someone to keep pinning
Encrypted originals	**Lost**	Cannot run detection
Encryption keys	**Lost**	Cannot decrypt originals
Watermark secret key	**Lost**	Cannot extract watermarks
Watermark extraction	**Dead**	Core value proposition dies

Table 18: Impact of NFTmarket Failure

15.2 Why There's No "Publish the Spec" Escape Hatch

The watermark uses a secret key to:

- Determine which frequency coefficients carry the data
- Generate the pseudorandom spreading sequence
- Scramble the embedded message

If this key were published:

- Anyone could extract watermarks (fine)
- Anyone could **forge** watermarks (disaster)
- Anyone could **remove** watermarks (disaster)

The secret key cannot be published. This means watermark extraction will **always** require the authority.

15.3 What Still Works Without the Authority

Even if NFTmarket disappears:

1. **"I own token #123"** – Query blockchain
2. **"Token #123 was minted on Jan 15, 2025"** – Block timestamp
3. **"The license agreement says X"** – Fetch from Arweave, verify hash
4. **"This exact file is token #123"** – Compare SHA-256 to on-chain imageHash

15.4 What Dies Without the Authority

1. **"This screenshot is from token #123"** – Can't extract watermark
2. **"This JPEG is a copy of my NFT"** – Can't trace transformed images
3. **"This repost is my art"** – No way to link image to token

The tokens don't become worthless – they become regular NFTs with nice legal documentation. But the watermark-based provenance, the thing that makes NFTmarket special, requires NFTmarket to keep operating.

15.5 IPFS Failure vs NFTmarket Failure

Two different failure modes:

IPFS Failure (Pinata stops pinning):

- IPFS preview gone from network
- But any copy in the wild **still contains the watermark**
- Find any copy → extract watermark → restore provenance
- **The watermark saves you**

NFTmarket Failure (company shuts down):

- Blockchain records safe
- Arweave permanent
- **Watermark extraction impossible** (secret key lost)
- Images become untraceable

The key difference:

- IPFS fails → Watermark saves you (any copy works)
- NFTmarket fails → Watermark dies with it (secret key lost)

16 Copyright and Legal Reality

16.1 AI Art Cannot Be Copyrighted

In the US (and increasingly elsewhere), copyright requires human authorship. The US Copyright Office has explicitly ruled that images generated by AI are not copyrightable.

Scenario	**Copyrightable?**
Human draws artwork	Yes
Human writes prompt, AI generates image	No
Human significantly modifies AI output	Maybe (only human parts)

Table 19: AI Art and Copyright

For AI art: The "Transfer" license is selling something that doesn't legally exist. Both parties can agree to treat it as copyrighted, but courts may not enforce it.

16.2 What the Watermark Actually Provides

The watermark is **evidence**, not **enforcement**.

It Proves	**It Doesn't Provide**
Which token an image came from	Automatic takedowns
Who minted first (timestamp)	Legal injunctions
Chain of custody	Damages recovery
That you own the NFT	Copyright (for AI art)

Table 20: What Watermarks Do and Don't Provide

The watermark helps you prove your case. You still have to fight it yourself.

16.3 The Opportunity for Physical Artists

A painter could:

1. Paint the artwork (copyright exists – human creation)
2. Photograph it (high quality digital capture)
3. Mint a watermarked NFT with "Transfer" license
4. Sell both together: physical painting + NFT

The buyer now receives:

- The physical painting (traditional sale)
- The NFT proving digital rights ownership
- Watermark protection against digital theft
- Signed bilateral agreement on-chain

This creates a legitimate market for reproduction rights that currently doesn't exist in the traditional art world.

17 Watermark Technical Details

17.1 The Watermarking API

NFTmarket uses a cloud-hosted watermarking service.

Endpoint: POST /watermark

Request:

- Header: `X-API-Key: <api-key>`
- Content-Type: `multipart/form-data`
- Body:
  - `image` (required): PNG or JPG file
  - `message` (required): Text to embed (`wallet:tokenId`)
  - `strength` (optional): 1-100, default 10

Response (SSE stream):

```
data: {"progress":"Loading image...","percent":10}
data: {"progress":"Embedding watermark (1/5)","percent":24}
data: {"progress":"Compressing image...","percent":90}
data: {"complete":true,"downloadUrl":"/download/{jobId}"}
```

17.2 Watermark Properties

Transformation	Survives?	Notes
JPEG compression	Yes	Designed for this
PNG to JPEG conversion	Yes	Format-agnostic
Resizing/scaling	Yes	Multi-resolution embedding
Screenshots	Yes	Minimal transformation
Social media upload	Usually	Depends on compression level
Color adjustments	Mostly	Embedded in luminance channel
Print and scan	Sometimes	Most challenging attack
Cropping	Partial	Depends on how much removed

Table 21: Watermark Robustness

17.3 The Robustness Trade-off

Watermarks balance three competing goals:

- **Robustness** – Survives transformations
- **Imperceptibility** – Can't be seen
- **Capacity** – How much data can be embedded

You can only optimize for two. NFTmarket prioritizes robustness and imperceptibility, embedding just enough data (wallet + tokenId) to establish provenance.

18 Future Directions: Zero-Knowledge Watermarks

Can we prove a watermark exists **without revealing the secret key**?

18.1 The Vision

Operation	Who Can Do It
Embed watermark	Authority only (has secret key)
Remove watermark	Authority only (knows where it is)
Verify watermark	**Anyone (via ZKP)**

Table 22: Zero-Knowledge Vision

This would break the central point of failure for *verification* while keeping *embedding/removal* secure.

18.2 How It Could Work

1. Authority publishes a **verification circuit** (the algorithm, not the key)
2. At mint time, authority generates a **proof** for each image
3. Proof stored on Arweave (permanent)
4. Anyone can verify the proof against an image
5. No secret key needed for verification

18.3 The Challenges

Circuit Complexity: Watermark extraction involves DCT transforms, correlation with pseudorandom sequences, and thresholding – expensive to encode as ZKP circuits.

Transformation Handling: ZKP proves "this exact image has this watermark." But watermarks survive transformation. A screenshot won't match the proof for the original.

New Minting Still Centralized: ZKP helps verification, not embedding. The authority is still needed to mint.

18.4 Current Status

For MVP, we accept the central point of failure and document it honestly.

Future research directions:

1. Asymmetric watermarking schemes (detection key $\neq$ embedding key)
2. ZKP for exact-match verification (proofs on Arweave)
3. TEE (Trusted Execution Environment) for transform-resistant verification

The "holy grail" – ZKP that handles arbitrary transformations – is an open research problem.

Summary: Key Takeaways

For the Watermarking Expert

1. **Your watermark is the cornerstone.** Without it, there's no link between images and tokens.
2. **The watermark embeds `wallet:tokenId`.** This uniquely identifies the token across the entire blockchain ecosystem.
3. **Both preview and original are watermarked.** Screenshots of the preview can be traced.
4. **The blockchain provides timestamps, not the watermark.** Priority disputes are resolved by block timestamps.
5. **Extraction requires your secret key.** This is a central point of failure, but necessary for security.
6. **The watermark survives transformation.** This is its key advantage over cryptographic hashes.

What Makes This Different

Regular NFT	NFTmarket
Owns token, not image	Owns token + embedded proof
Screenshot = no connection	Screenshot = traceable
"Right-click save" wins	Watermark survives
No legal agreement	Bilateral signed license
No copyright transfer	Explicit license types

Table 23: Regular NFT vs NFTmarket

The Trade-offs

Benefit	Cost
Traceable provenance	Depends on watermarking authority
Clear license agreements	2.5% platform fee
Bilateral signatures	More complex UX
Copyright can be sold	Legal enforcement still required

Table 24: Benefits and Trade-offs

This document was created to explain NFTmarket to a watermarking expert.

For technical implementation details, see the individual documentation files in `/docs/`.